

Inconsistency Detection in Natural Language Requirements using ChatGPT: a Preliminary Evaluation

^{1st} Alessandro Fantechi

*Dipartimento di Ingegneria dell'Informazione
Università di Firenze
Firenze, Italy, alessandro.fantechi@unifi.it*

^{3rd} Lucia Passaro

*Dipartimento di Informatica
Università di Pisa
Pisa, Italy, lucia.passaro@unipi.it*

^{2nd} Stefania Gnesi

*Istituto di Scienza e Tecnologie dell'Informazione "A.Faedo"
Consiglio Nazionale delle Ricerche, ISTI-CNR
Pisa, Italy, stefania.gnesi@isti.cnr.it*

^{4th} Laura Semini

*Dipartimento di Informatica
Università di Pisa
Pisa, Italy, laura.semini@unipi.it*

Abstract—With the rapid advancement of tools based on Artificial Intelligence, it is interesting to assess their usefulness in requirements engineering. In early experiments, we have seen that ChatGPT can detect inconsistency defects in natural language (NL) requirements, that traditional NLP tools cannot identify or can identify with difficulties even after domain-focused training.

This study is devoted to specifically measuring the performance of ChatGPT in finding inconsistency in requirements. Positive results in this respect could lead to the use of ChatGPT to complement existing requirements analysis tools to automatically detect this important quality criterion. For this purpose, we consider GPT-3.5, the Generative Pretrained Transformer language model developed by OpenAI. We evaluate its ability to detect inconsistency by comparing its predictions with those obtained from expert judgments by students with a proven knowledge of RE issues on a few example requirements documents.

Index Terms—ChatGPT, Natural Language Requirements, Inconsistency Detection.

I. INTRODUCTION

Software requirements are normally expressed informally through natural language sentences. On the one hand, ambiguity is an intrinsic characteristic of natural language and it is not rare that sentences have multiple alternative interpretations. Second, requirements are often a product of collaborative work, indeed they are prone to include inconsistency between each other, as well as different levels of specificity and details. Inconsistency can be considered as a situation in which two or more requirements disagree with each other and contain conflicting or contradictory descriptions of the expected behaviour of the system under specification.

Managing inconsistency is a key challenge in requirements engineering (RE) and several strategies have been developed

in the RE community to help analysts identify conflicting requirements [8].

An example strategy is to group requirements by similarity or using glossary terms, and looking for conflicts in each cluster, which contains related requirements. In fact, conflicts more likely occur between related requirements. Tools have been developed to help in the clustering step, see e.g. [1].

An alternative approach is the one of Zowghi and Gervasi [7] showing how to transform a set of NL requirements into a corresponding set of logical statements and then apply a theorem prover or a model checker to find inconsistent statements.

An inconsistency detection technique on not fully formalized, but structured, requirements is found in the aspect-oriented approach to requirements analysis [2].

The need for automatic tools to detect inconsistency is advocated by Çevikol and Aydemirin [5] that plan to employ NLP and machine learning techniques to extract domain and requirements models from the NL requirements and check for inconsistency using these models.

In this paper, we focus on inconsistency detection on Natural Language requirements by means of AI-based tools. Indeed, the rise of Large Language Models (LLMs) and their application in AI-based tools open the way to new in-depth automatic analysis of Natural Language requirements [6]. We decided to exploit at this regard one of the larger LLMs available, namely ChatGPT¹, a chatbot developed by OpenAI and built on top of OpenAI's GPT3.5 model [3]. ChatGPT garnered a lot of attention from both research and industrial communities because it can solve a lot of tasks concerning a wide array of domains of knowledge. OpenAI's ChatGPT interacts in a conversational way, by answering both direct and follow-up questions, highlighting incorrect premises, and

¹<https://openai.com/blog/chatgpt>

The research has been partially supported by the MIUR, Italy project PRIN 2017 FTXR7S "IT-MaTTeR" (Methods and Tools for Trustworthy Smart Systems). We are thankful to the students who participated in the experiments.

rejecting some inappropriate requests. ChatGPT is based on a Transformer architecture which is becoming the preferred model for NLP problems [9]. Its very first model was trained using supervised fine-tuning, and then it has been improved by means of Reinforcement Learning on Human Feedback.

As far as we know, there is still no documentation or literature on the inconsistency detection capabilities of ChatGPT. We hence decided to challenge ChatGPT to automatically identify inconsistency and compare its performances with the ones provided by humans having expertise in this field.

In this respect, this paper addresses the following Research Question:

RQ1 What are the limitations and challenges associated with using ChatGPT to detect inconsistencies in requirements?

RQ2 How does the ChatGPT performance for inconsistency detection compare to expert judgment?

In this paper we present preliminary experiments aimed at assessing whether ChatGPT can detect inconsistency defects in natural language (NL) requirements, that traditional NLP tools cannot identify or can identify with difficulties even after domain-focused training.

The scope of the experiments is limited to four requirements documents. We have perturbed the original documents adding mutant requirements: these are new requirements built to be conflicting with the original ones. We then asked volunteer students to both manually analyze and prompt ChatGPT to find the inconsistent pairs in the mutated documents. Note that in this experiment we limit to the detection of pairs of inconsistent requirements, ignoring other sources of inconsistency, and we will often use for short the term *inconsistencies* to refer to pairs of inconsistent requirements.

The paper is organized as follows. Section II describes data preparation: the example requirements documents and mutated documents to be used in the experience. The analysis of the data returned by the students is addressed in Section III. The final sections deal with discussion and lessons learned, threats to validity, and conclusions.

II. DATA PREPARATION

In order to accomplish our experience, we have taken some requirement documents and we have injected conflicting requirements into them. Precisely:

a) *Identification of Requirements data*: We identified four datasets for conducting our experiments. In particular, we used: **Coffee machine** that gives few requirements of an automatic coffee vending machine; **E-shop** that describes a simple online shopping system; **Library**, that describes the requirements for the System Administration Module of an urban library system; **ETCS**, which defines the functional requirements of the European Train Control System, which provides the driver with the necessary information for safe train driving.

In Table I we summarise some characteristics of the considered documents. For the purposes of the present work and in particular for the construction of ground truth, an

TABLE I
ORIGINAL REQUIREMENT DOCUMENTS: NUMBER OF REQUIREMENTS, AUTHORSHIP AND CHARACTERISTICS OF THE SYSTEM TO BE.

	reqs	issued by	characteristics
Coffee_mach	6	authors	toy example
E-shop	18	authors	toy example
Library	94	company	information system
ETCS	49	agency	control system

important property of these documents is that they are all already validated and free from inconsistency.

b) *Anonymization and generation of the mutants*: We removed any enumeration or identification label from the requirements and we added 15 new requirements to each document. The number 15 was chosen so as to have a document (Coffee Machine) with more injected requirements than the originals, one (E-shop) in which they were in a comparable number, and two documents (Library and ETCS) in which the injected requirements were less than the original ones.

The new requirements were generated by ChatGPT itself, with the prompt “can you generate fifteen new example requirements that introduce an inconsistency wrt the following ones? < requirements_list >”. It is important to note that the ChatGPT account used for the generation of conflicting requirements has not been used for detection.

We used ChatGPT as a generator to introduce broader and more unexpected inconsistencies than those generated by humans. While humans can introduce inconsistencies based on common patterns and biases, a language model such as ChatGPT has the potential to introduce new and different forms of inconsistencies.

c) *Mutated requirements documents*: For each original document, we created a new *mutated requirements document* as follows. We added the new requirements to the original list, we shuffled them and we added a consistent numbering (i.e., by adding an incremental number to the shuffled list). Moreover, we maintained an additional document where each requirement was provided with a label indicating whether it was original (marked with 1) or an automatically generated (marked with 0). In Tables II and III we present the original and mutated requirements of the coffee machine, respectively.²

d) *ChatGPT prompt definition*: To query ChatGPT we have given in input the following prompt, which includes the list of mutated requirements in text form: “Consider the following requirements. Identify the pairs conflicting with each other. Provide me with all the possible inconsistent pairs formatted as follows: “1,3”; “5,8” < mutated_requirements_list >.”

e) *Google Forms module*: Finally, we built a Google forms module asking, for each mutated requirements document: a) the set of conflicting pairs resulting from manual analysis b) a ChatGPT answer to the prompt defined above. Overall, the forms have been filled in by 9 volunteers. All of them are students in Computer Science, with experience in the field of requirement engineering.

²All the documents are available at <https://zenodo.org/record/8089810>.

TABLE II
COFFEE-MACHINE ORIGINAL REQUIREMENTS

C1	After inserting a suitable coin, the user shall choose a beverage and select the amount of sugar.
C2	The machine shall offer, as beverages, Coffee and Cappuccino or Tea.
C3	The machine shall always offer coffee.
C4	A ringtone possibly has to be played after beverage delivery.
C5	After the beverage is taken, the machine returns idle.
C6	The British market requires tea and excludes any ringtone.

TABLE III
COFFEE-MACHINE MUTATED REQUIREMENTS

1)	The British market requires tea and excludes any ringtone.
2)	The machine shall offer a selection of alcoholic beverages.
3)	The machine shall offer, as beverages, Coffee and Cappuccino or Tea.
4)	The machine shall not return to idle after the beverage is taken.
5)	The machine shall offer hot chocolate as a beverage option.
6)	The machine shall only offer Coffee as a beverage option in the British market.
7)	The machine shall not offer Cappuccino as a beverage option.
8)	After inserting a suitable coin, the user shall choose a beverage and select the amount of sugar.
9)	The machine shall only offer coffee on odd-numbered days and tea on even-numbered days.
10)	The machine shall always play a ringtone after beverage delivery, even if the user has selected Tea as their beverage.
11)	A ringtone possibly has to be played after beverage delivery.
12)	The user shall choose a beverage and select the amount of cream instead of sugar.
13)	The user shall not be able to select the amount of sugar in their beverage.
14)	The machine shall offer, as beverages, Coffee and Cappuccino or Hot Chocolate.
15)	The machine shall never return to an idle state after the beverage is taken. Instead, it shall continue to display advertisements until the user interacts with it again.
16)	After the beverage is taken, the machine returns idle.
17)	The machine shall play a ringtone before beverage delivery on weekdays and not on weekends.
18)	The machine shall always offer coffee.
19)	The machine shall not offer Cappuccino as a beverage option.
20)	The user shall choose a beverage and select the amount of sugar before inserting a coin.
21)	The machine shall return to the main menu after beverage delivery on weekends and not on weekdays.

f) *Data Collection*: All students have run ChatGPT, while only a subset of them performed the manual analysis. Answers were collected in a spreadsheet as shown in Figure 1. The full data set is available in the Zenodo archive.

III. DATA ANALYSIS

We performed the following steps to analyse the results:

a) *Ground truth construction*: We have defined, in a joint meeting, the reference pairs to be used as ground truth. To this end we analysed the stored files where injected requirements were marked. Each injected requirement has been compared with all the original ones and also with the other injected ones, looking for conflicts.

We have put the ground truth pairs into a spreadsheet to be used for analysis. For example, the pair (8,13) in Table III is an obvious inconsistency.

We can notice that within the requirements not all inconsistencies have the same degree of strength. Beside *strong* inconsistencies, where one requirement expresses exactly the opposite of the other, such as (8,13), we can find *weak* inconsistencies, such as pairs of sentences one implying the other, i.e. whose logical meaning is included one in the other, but not vice-versa. An example is the pair (10;11) in Table III, since all machines satisfying 10 satisfy 11 as well, but not the other way round. Another weak inconsistency is the pair (3;14): each requirement admits an implementation not admitted by the other, but there exists anyway an implementation (a machine offering Coffee and Cappuccino) satisfying both. We have labeled "W" these weakly conflicting pairs in the ground truth. The ground truth for the coffee machine example is shown in the table at the bottom left of Fig. 1. For each document, we have hence the number of *ground truth pairs* (excluding weak) and the number of *weak ground truth pairs*.

Supplementary material including both data and annotations is provided in <https://zenodo.org/record/8089810>.

IV. PRELIMINARY RESULTS

We provide a quantitative analysis, based on the following performance metrics, where *tp* are true positives, *fp* are false positives and *fn* are false negatives.

$$precision = \frac{tp}{tp + fp} = \frac{|found\ pairs \cap ground\ truth\ pairs|}{|found\ pairs \setminus weak\ ground\ truth\ pairs|}$$

$$recall = \frac{tp}{tp + fn} = \frac{|found\ pairs \cap ground\ truth\ pairs|}{|ground\ truth\ pairs|}$$

In this paper quantitative analysis is focused on strong inconsistencies: weak inconsistencies are considered neither true positives nor false positives. We anyway keep track of weak inconsistencies in the discussions and we are aware that they require a more in-depth analysis, left for future work.

Let the number of pairs found by a student be $tp + fp + wp$ (weak positive), precision hence is calculated as the ratio between *tp* (the number of pairs found that are also *ground truth pairs*) and $tp + fp$ and that is, the number of pairs found that are not in the set *weak ground truth pairs*. Recall is also calculated by ignoring weak positives, both at the numerator (pairs found) and the denominator (ground truth).

In order to obtain a reliable evaluation, for each document, and for each obtained answer on that document, we calculated the following metrics: *precision_manual_analysis*, *precision_chatGPT*, *recall_manual_analysis*, *recall_chatGPT*. Finally, for each group of requirements (i.e., document) we calculated the minimum, maximum and average value for each metric. In the following subsection, we report on the results obtained for each document.

A. Coffee machine

We first consider the coffee machine case study, which is short enough to allow us to fully report the results. The original document is a tiny one, but the mutated document contains many inconsistencies and deserves a detailed treatment. Table IV shows overall results, while Figure 1 shows the details of the analysis: ChatGPT has been asked the same prompt from

9 different accounts. Only three expert students volunteered for manual analysis.

TABLE IV
COFFEE MACHINE: PERFORMANCE MEASURES.

	manual		ChatGPT	
	precision	recall	precision	recall
max	1,00	0,55	0,75	0,30
min	0,60	0,15	0,25	0,05
average	0,82	0,30	0,45	0,12

B. E-shop

Our second experience involved the E-shop example: the performance values are given in the table V and show that ChatGPT performs almost as well as the experts, with a difference of approximately 12% in precision and 16% in the recall. Notably, in this document, we injected 15 mutants in a document originally containing 18. The rate of injected requirements over original ones is lower than in the coffee example, making this case more realistic.

TABLE V
E-SHOP: PERFORMANCE MEASURES.

	manual		ChatGPT	
	precision	recall	precision	recall
max	1,00	0,76	1,00	0,76
min	0,89	0,47	0,79	0,18
average	0,96	0,63	0,85	0,53

C. Library and ETCS

We discuss Library and ETCS case studies together because they both highlighted a ChatGPT weakness, namely the inability to analyze large input documents. Indeed, ChatGPT has analysed roughly the first 52 (out of 108) requirements of Library and the first 38 (out of 54) of ETCS. The performances values are affected by the ChatGPT limitation and are shown in Table VI and Table VII.

One particular false positive pair detected by ChatGPT in 6 answers (out of 8) in the Library case study, is formed by the two requirements: *The System Administration Module shall only be accessible through a Mac-compatible client* and *The System Administration Module shall be accessible only through a macOS-compatible client*. The only hypothesized reason for this false positive is that ChatGPT considers Mac and macOS not nearly synonyms, as they are.

TABLE VI
LIBRARY: PERFORMANCE MEASURES.

	manual		ChatGPT	
	precision	recall	precision	recall
max	0,94	0,80	0,50	0,15
min	0,29	0,10	0,00	0,00
average	0,71	0,54	0,32	0,09

V. DISCUSSION

The data gathered from the experiences allow us to answer, although in an initial way, the RQs:

TABLE VII
ETCS: PERFORMANCE MEASURES.

	manual		ChatGPT	
	precision	recall	precision	recall
max	1,00	0,56	0,45	0,15
min	0,79	0,53	0,20	0,06
average	0,90	0,54	0,29	0,09

a) *RQ1: What are the limitations and challenges associated with using ChatGPT to detect inconsistencies in requirements?*: We observe that ChatGPT fully processes only short requirement documents; when asked to analyze large documents, it only considers the first part. The detection of inconsistency depends indeed on the processing of the document as a whole; however, a solution that approximates the analysis of the entire requirements document is to divide it into simple clusters of similar requirements and analyze the clusters separately.

Since ChatGPT is based on training data and complex learning algorithms instead of clearly defined rules, it is not possible to know exactly why it returns a certain result. This is referred to as *lack of explainability and transparency*. Moreover, it suffers from the *non-reproducibility* issue: it is rare to get the same answer twice. These are well-known issues that need to be considered when using the bot, particularly when a quality assessment of requirements is sought. Nevertheless, it is not rare to obtain different annotations from humans as well, if they are asked to repeat a task many times.

b) *RQ2: How does the ChatGPT performance for inconsistency detection compare to expert judgment?*: All experiences show that the experts perform better in all case studies both from the point of view of precision and of recall. However, we can observe that manual detection of inconsistencies is more expensive in terms of manpower: a document with n requirements is fully analyzed after looking into $\frac{n(n-1)}{2}$ pairs, whereas examining the ChatGPT answers to distinguish true from false positives is a much lighter task. Consider, for example, the case of the coffee machine: a full manual search asks for analyzing 190 pairs of requirements, while the average number of ChatGPT responses, to be reviewed, was 7,44.

The data collected with E-shop are overall very much in favor of using ChatGPT: comparable performances in terms of precision and recall and only an average of 12,4 chatbot responses to be reviewed (vs 496 pairs).

Finally, automatic analysis has found conflicts that some expert judges did not: for instance, the pair (1;6), of the mutated coffee machine (see Table III) was consistently found by ChatGPT (7 out of 8 answers), while it was only found by 1 out of 3 experts. On the other hand, the pair (4;16) had been found by all experts and by ChatGPT in only one run. Other examples can be found in the Zenodo archive.

It is important to notice additional aspects emerging from our analysis. First, the number of perturbations injected plays a role in the overall performances in ChatGPT. In fact, E-shop and Coffee Machine obtain different degrees of performance in inconsistency detection.

TABLE VIII
LINGUISTIC CHARACTERISTICS OF THE REQUIREMENT DOCUMENTS.

	Features	Library	E-shop	CoffeeMachine	ETCS
Text Properties	n_sentences	120.0	35.0	23.0	79.0
	n_tokens	2471.0	598.0	356.0	1701.0
	tokens_per_sent	20.59	17.09	15.48	21.53
Lexical Properties	lexical_density	0.64	0.49	0.5	0.52
Syntactic Properties	avg_max_depth	4.38	3.89	3.52	4.44
	avg_token_per_clause	8.86	8.93	9.37	8.3

proper nouns, verbs, adjectives, adverbs) over the total number of words in the document (lexical_density). Syntactic properties refer to the following metrics: *avg_max_depth* is the mean of the maximum tree depths extracted from each sentence of a document; *avg_token_per_clause* refers to the average clause length, computed in terms of the average number of tokens per clause. It is important to notice that the various requirements have a different linguistic profiling, that can differently impact on the final performances obtained with ChatGPT. Notably, the longer documents in terms of number of sentences, tokens and average tokens (library and ETCS) have also a more complex syntactic structure (e.g. the higher average depth of the syntactic trees) and a higher lexical density.

Overall, we can observe that length and complexity from both lexical and syntactic perspectives negatively affects the performance of the Language Model. The linguistic analysis provides only partial insights. An interesting future work could be to further investigate the role of linguistic features of documents with respect to ChatGPT behavior. However, this analysis requires a more balanced dataset from both the class perspective (inconsistent vs. correct pairs) and the linguistic perspective (number of sentences, tokens, etc.).

It will also be interesting to compare the performance of ChatGPT to those of other related approaches [2], [7].

Since ChatGPT shows a limitation on the size of the input document, reflected in a low performance in the case of large documents, in the future we plan to use it in combination with clustering tools (e.g. REGICE [1]), to reduce the size of the requirement sets to be analyzed together. We also expect that LLM tools abilities will improve in the next years, as already witnessed by GPT-4.0, w.r.t. to GPT-3.5 used in this study.

We also plan a systematic analysis of balanced sets of requirements. This will enable us to disentangle the impact of several aspects on the final performances, such as language complexity and length and number of conflicts present.

Finally, future work may widen the assessment to include the detection of redundancies, requirements at a different level of specificity, and requirements that are weakly conflicting. Even if these are not strictly inconsistent from a logical point of view, they must be identified and reformulated to obtain a quality requirements document.

VIII. CONCLUSIONS

GPT-like Large Language Models are the most powerful ones to date. The recently developed chatbot makes it queryable out-of-the-box, without having to write code, thus

making it accessible for a wide array of tasks involving human language and (basic) reasoning.

The aim of this work was a preliminary investigation into whether ChatGPT is reasonably suitable for detecting inconsistency in software requirements, to be taken into consideration for speeding up the process of refining requirements in the field of software development. On the one hand, the results indicate that it cannot substitute expert judgment. On the other hand, our experience demonstrated it is definitely worth using in order to complement manual analysis as well as to speed up human annotation.

REFERENCES

- [1] C. Arora, M. Sabetzadeh, L. C. Briand, and F. Zimmer. Automated extraction and clustering of requirements glossary terms. *IEEE Trans. Software Eng.*, 43(10):918–945, 2017.
- [2] I. S. Brito, A. Moreira, R. A. Ribeiro, and J. Araújo. Handling conflicts in aspect-oriented requirements engineering. In A. Moreira, R. Chitchyan, J. Araújo, and A. Rashid, editors, *Aspect-Oriented Requirements Engineering*, pages 225–241. Springer, 2013.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [4] D. Brunato, A. Cimino, F. Dell’Orletta, G. Venturi, and S. Montemagni. Profiling-UD: a tool for linguistic profiling of texts. In *Proc. of the 12th Language Resources and Evaluation Conference*, pages 7145–7151, Marseille, France, May 2020. European Language Resources Association.
- [5] S. Çevikol and F. B. Aydemir. Detecting inconsistencies of natural language requirements in satellite ground segment domain. In *Joint Proceedings of REFSQ-2019 Workshops, Doctoral Symposium, Live Studies Track, and Poster Track co-located with the 25th Int. Conf. on Requirements Engineering: Foundation for Software Quality (REFSQ 2019), Essen, Germany, March 18th*, volume 2376 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2019.
- [6] A. Fantechi, S. Gnesi, and L. Semini. Rule-based NLP vs ChatGPT in ambiguity detection, a preliminary study. In *Joint Proc. of 28th Int. Conf. on Requirements Engineering: Foundation for Software Quality REFSQ-2023 Workshops, Doctoral Symposium, Posters & Tools Track and Journal Early Feedback: NLP4RE, Barcelona, Spain, April 17-20, 2023*, volume 3378 of *CEUR Workshop Proceedings*.
- [7] V. Gervasi and D. Zowghi. Reasoning about inconsistencies in natural language requirements. *ACM Trans. Softw. Eng. Methodol.*, 14(3):277–330, 2005.
- [8] G. Spanoudakis and A. Zisman. Inconsistency management in software engineering: Survey and open research issues. In *Handbook of Software Engineering and Knowledge Engineering: Volume I: Fundamentals*, pages 329–380. 2001.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.